

Pruebas de rendimiento con Locust

Taller — Ingeniería de Software II

Docente: Albeiro Montes

Ingeniería de Software II

8 de octubre de 2026

Agenda

- 1 Contexto del taller
- 2 Tipos de prueba
- 3 API de laboratorio
- 4 Ejecución de la API — paso a paso
- 5 Locust — maqueta del entregable
- 6 Métricas e informe
- 7 Cierre del taller

Propósito de la sesión

- Diferenciar pruebas de **carga**, **estrés** y **capacidad** en servicios REST.
- Desplegar y verificar la API de laboratorio (Laravel 9).
- Diseñar, implementar y ejecutar escenarios Locust (responsabilidad del equipo).
- Registrar métricas y conclusiones en un informe técnico breve.

Modalidad

El material incluye la API, datos sembrados y lineamientos. La construcción del `locustfile.py` y la parametrización de escenarios **no** se entregan resueltas; forman parte de la evaluación del taller.

- Herramienta en Python para simular clientes concurrentes.
- Cada cliente ejecuta **tareas** repetidas sobre HTTP(S).
- Interfaz web para controlar usuarios simulados y spawn rate.
- Reportes de latencia, throughput y tasa de error por operación.

Términos

Host: URL base del servicio

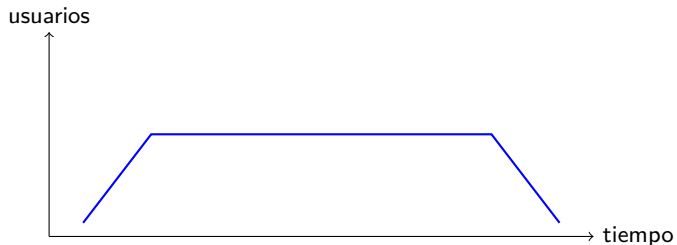
Users: concurrencia

Spawn rate: usuarios/s al arrancar

RPS: peticiones por segundo

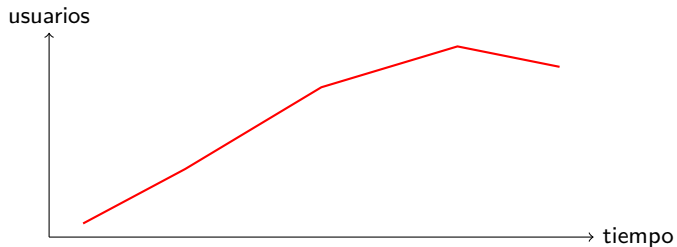
Prueba de carga

- Tráfico esperado (o ligeramente superior) de forma **sostenida**.
- Objetivo: verificar cumplimiento de SLAs acordados (latencia, errores).
- Duración orientativa: 10–30 minutos en régimen estable.



Prueba de estrés

- Incremento progresivo de carga **más allá** del diseño nominal.
- Objetivo: identificar punto de degradación y tipo de fallo (timeouts, 5xx).
- Debe registrarse el nivel de concurrencia/RPS asociado al quiebre observado.



Prueba de capacidad (soak)

- Carga alta **sostenida** durante un periodo prolongado.
- Objetivo: detectar degradación gradual (memoria, pool de conexiones, I/O).
- Duración orientativa: 60 minutos o más, según indicación del docente.

Síntesis

Carga: comportamiento nominal **Estrés:** límite del sistema **Capacidad:** resistencia en el tiempo

Descripción general

- API REST Laravel 9, sin autenticación (entorno académico controlado).
- Tabla `users` con `birth_date`; volumen de referencia: 1,500,000 registros tras el seed.
- Cuatro operaciones HTTP documentadas en el repositorio del curso.
- Los GET iniciales devuelven el conjunto completo de filas (`total + data`); se trata de una **mala práctica** intencional para análisis.

Contrato de endpoints

Método	Ruta	Descripción
GET	/api/users	Listado de usuarios del sistema
GET	/api/users/emails	Identificador y correo electrónico
GET	/api/users/over-twenty	Usuarios con edad superior a 20 años (según birth_date)
POST	/api/users/bulk	Creación simultánea de tres usuarios en un mismo cuerpo JSON

Sugerencia (no obligatoria en la fase inicial)

Se recomienda **modificar la API** para paginar las respuestas GET (page, per_page, límite máximo), antes de ejecutar Locust contra el dataset completo. Documentar el impacto en el informe.

Paso 1 — Requisitos del entorno

- 1 PHP 8.0 o superior, Composer y motor de base de datos (MySQL o PostgreSQL recomendado).
- 2 Clonar o descomprimir la carpeta `laravel-api` provista en el taller.
- 3 Instalar dependencias en el directorio del proyecto.

```
cd laravel-api
composer install
copy .env.example .env
php artisan key:generate
```

Ajustar credenciales de base de datos en el archivo `.env` antes de migrar.

Paso 2 — Base de datos y seed

- ❶ Crear la base de datos vacía en el servidor SQL.
- ❷ Ejecutar migraciones y seed (incluye MassUserSeeder vía DatabaseSeeder).
- ❸ Verificar variables opcionales `MASS_USER_SEED_COUNT` y `MASS_USER_CHUNK_SIZE` si se desea reducir volumen en pruebas locales.

```
php artisan migrate:fresh --seed
```

El proceso puede demorar varios minutos con 1.5 millones de registros.

Paso 3 — Servidor y verificación manual

- 1 Levantar el servidor de aplicación en el puerto acordado.
- 2 Probar cada endpoint con cliente HTTP (navegador, Postman, curl) **antes** de Locust.
- 3 Confirmar códigos de estado y estructura JSON esperada.

```
php artisan serve --host=0.0.0.0 --port=8000
```

URL base de la API: `http://HOST:8000/api`. Registrar la URL en la documentación del equipo.

Paso 4 — Validación funcional mínima

- GET /users: debe responder con total y arreglo data.
- GET /users/emails: mismo esquema, campos reducidos.
- GET /users/over-twenty: incluye cutoff_date y usuarios filtrados.
- POST /users/bulk: cuerpo con arreglo users de longitud 3; correos únicos; respuesta 201 si la validación es correcta.

Observación pedagógica

Con el volumen completo del seed, un GET sin paginar puede agotar memoria o tiempo de respuesta. Ese comportamiento debe mencionarse en el informe y, como **sugerencia**, abordarse mediante refactor del controlador.

Archivos que debe elaborar el equipo

- `locustfile.py` — definición de usuarios simulados y tareas HTTP.
- `requirements.txt` o entorno virtual con dependencia Locust (versión acordada).
- Informe del taller: escenarios, parámetros, capturas o export HTML, análisis.

Restricción

En esta presentación **no** se incluye el código interno de las pruebas. Solo se describe la **maqueta** y los criterios de aceptación.

Maqueta lógica de locustfile.py

Bloque	Indicación (a implementar por el equipo)
Configuración	URL base vía argumento <code>-host</code> ; tiempo de espera entre peticiones acorde al escenario.
Clase de usuario	Una clase que represente al cliente HTTP; consultar documentación oficial de Locust.
Tarea: listado	Petición GET al endpoint de usuarios; ponderación acorde al perfil de tráfico.
Tarea: correos	Petición GET al endpoint de correos.
Tarea: edad	Petición GET al endpoint <code>over-twenty</code> .
Tarea: alta lote	Petición POST al endpoint <code>bulk</code> con tres registros válidos; estrategia para correos no duplicados bajo carga.
Validación	Marcar fallos cuando el código HTTP o el cuerpo no cumplan lo esperado (documentación Locust: respuestas y eventos).

Escenarios — lineamientos (sin código)

- ① **Carga:** mezcla de GET predominante y POST minoritario; concurrencia moderada; duración en meseta; SLAs definidos por el equipo y validados con métricas p95 y tasa de error.
- ② **Estrés:** misma mezcla de operaciones; rampa de usuarios agresiva; registro del punto en que aumentan latencias o errores 5xx/timeouts.
- ③ **Capacidad:** carga cercana al máximo **estable** (no al quiebre); ejecución prolongada; análisis de tendencia temporal.

Cada escenario debe documentar: usuarios simulados, spawn rate, duración, host, versión de Locust y estado de la API (con o sin paginación).

Ejecución de Locust — referencia operativa

- Instalar Locust en entorno virtual aislado.
- Invocar la herramienta indicando el archivo elaborado por el equipo y el host de la API.
- Interfaz web local para iniciar y detener pruebas (puerto por defecto documentado en Locust).
- Modo headless opcional para exportar reporte HTML en entregas automatizadas.

```
pip install locust
locust -f locustfile.py --host=http://HOST:8000
```

Los parámetros numéricos (-u, -r, -t, etc.) corresponden definirlos e justificarlos en el informe.

Datos a registrar

Por operación HTTP

- Percentiles de latencia
- Peticiones totales y RPS
- Porcentaje de fallos

Entorno

- CPU/RAM del servidor API
- Motor y tamaño del dataset
- Cambios aplicados a la API (si los hubo)

Entregables sugeridos

Repositorio o paquete con `locustfile.py`, evidencias de ejecución e informe PDF con comparación entre escenarios y reflexión sobre paginación e índices.

Errores frecuentes a evitar

- Omitir el parámetro `-host` o apuntar a puerto incorrecto.
- Contabilizar como éxito respuestas 422 en POST por correos duplicados.
- Ejercitar un solo endpoint y omitir la mezcla acordada.
- Confundir usuarios Locust con RPS (depende del tiempo entre tareas).
- Ejecutar 1.5M registros en SQLite en hardware limitado sin interpretar el contexto.

- Taller Ingeniería de Software II — docente Albeiro Montes.
- API Laravel desplegada según pasos 1–4; datos masivos vía `migrate:fresh -seed`.
- Tres tipologías de prueba: carga, estrés, capacidad.
- Locust: maqueta y criterios entregados; **implementación a cargo del equipo**.
- **Sugerencia:** refactor de GET con paginación antes de mediciones representativas.

- Locust: docs.locust.io
- Laravel 9: laravel.com/docs/9.x
- Material del curso: `laravel-api/`, `README-API-LOCUST.md`

Fin del taller — ¡Éxitos!